

Software Requirements Specification (SRS)

Online Food Ordering System

Spring Boot | Java | OOP

Team Members

MD Anisur Rahman — 2022331036
Abdullah Omar Sayeem — 2022331058
Angkon Roy — 2022331060
Muhammad Mahir Asef — 2022331064
Syed Rafi Alam — 2022331086

1. Introduction

This SRS describes the requirements for the Online Food Ordering System (OFOS) — a backend-centric web application built with Java and Spring Boot. The system allows customers to browse restaurant menus, place orders, and track deliveries, while restaurant staff and administrators manage operations through a secure backend.

2. Overall Description

2.1 Project Description

The OFOS is developed using Java, OOP principles, and Spring Boot, following a layered architecture (Controller → Service → Repository). An optional frontend may use ReactJS and Tailwind CSS, but the core focus is on backend logic, transaction handling, and data consistency.

2.2 Objectives

- Design a scalable, modular backend system using Spring Boot and OOP.
- Model real-world food ordering workflows: menu browsing, cart management, checkout, and order tracking.
- Ensure data consistency and reliability during transactions such as order placement and payment.
- Apply standard software engineering practices including requirement analysis, UML modeling, and layered architecture.
- Implement design patterns: Strategy (payment), State (order lifecycle), and Factory (object creation).
- Develop a role-based admin panel for managing restaurants, menus, and orders.

2.3 Stakeholders

- Customers — browse menus, place and track orders.
- Restaurant Staff — manage menus and process incoming orders.
- Administrators — monitor transactions and manage user roles.
- Development Team — design, build, and test the system.
- Course Instructor / Evaluator — reviews deliverables against academic requirements.

3. Functional Requirements

3.1 Customer

- Register and log in with email and password.
- Browse available restaurants and their menus.
- Search and filter menu items by name or category.
- Add, remove, and update items in a shopping cart.
- Place a food order and select a payment method (Bkash or Credit Card).
- Receive order confirmation after successful payment.
- Track order status in real time (Placed → Preparing → Out for Delivery → Delivered).

3.2 Restaurant Staff

- Log in and access the restaurant's management panel.
- Add, edit, and delete menu items (name, description, price, category, availability).
- View incoming orders and update their status through the lifecycle.
- Cannot access other restaurants' data or admin functions.

3.3 Administrator

- Log in with elevated privileges.
- View and filter all system transactions (by date, status, payment method).
- Manage user accounts: create, deactivate, and assign roles (Customer, Restaurant Staff, Admin).
- Cannot delete the only remaining Admin account.

4. Non-Functional Requirements

4.1 Security

- All passwords stored using BCrypt hashing.
- Authentication via JWT tokens (expire after 24 hours).
- Role-based access control enforced server-side via Spring Security.
- All API inputs validated and sanitized; SQL injection prevented via JPA parameterized queries.

4.2 Performance

- Menu browsing API response time: under 500 ms under normal load.

- Order placement: completed end-to-end in under 1 second.
- System should support at least 50 concurrent users.

4.3 Reliability

- Order and payment operations must be atomic using Spring @Transactional.
- Invalid order state transitions must be rejected without corrupting data.
- Exceptions must be logged with enough detail for debugging.

4.4 Maintainability

- Code must follow layered architecture with clear separation of concerns.
- Design patterns (Strategy, State, Factory) implemented following Open/Closed Principle.
- Core service logic must have unit tests using JUnit 5 and Mockito.

5. System Constraints and Assumptions

- Backend must be implemented in Java using the Spring Boot framework.
- OOP principles (encapsulation, inheritance, polymorphism, abstraction) must be applied throughout.
- External payment gateways (Bkash, Credit Card) are mocked for demonstration purposes.
- A relational database (MySQL or PostgreSQL) is used for persistent storage.
- The system will be evaluated alongside UML diagrams (Use Case, Class, Sequence, ER) prepared separately.
- All team members have access to a shared Git repository for version control.